ARTIFACT 5 OF 10 · ORANGEHRM QA ENGAGEMENT

Low-Level Design (LLD): Login & PIM Modules

Project: OrangeHRM Demo Automation **Author:** ABHISHEK K M **Version:** 1.0

> Interfaces only — no implementation bodies. This document is the contract the build phase implements. Signatures shown in Playwright-flavoured JavaScript; the Cypress implementation keeps identical class/method names with tool-idiomatic bodies (sync command chains, no `async`).

---

1. Login module

1.1 Page object class structure

```
BasePage
 └── LoginPage
DashboardPage  (assertion target after successful login — owned by Dashboard LLD,
                referenced here only for the post-login handoff)
```

`LoginPage` is deliberately flat — one screen, one class. The forgot-password screen is small enough to live as a section of `LoginPage` rather than its own class; if it grows, extract `ForgotPasswordPage` later.

1.2 LoginPage — method signatures

```
class LoginPage extends BasePage {
  async goto()                              // open /auth/login, wait for form visible
  async enterUsername(username)             // fill username field
  async enterPassword(password)             // fill password field
  async submit()                            // click Login button
  async loginAs({ username, password })     // goto + fill both + submit
  async getErrorMessage()                   // → string: banner text, or null
  async getFieldValidationMessages()        // → string[]: per-field "Required" messages
  async isLoginFormVisible()                // → boolean
  async clickForgotPassword()               // navigate to reset screen
  async getForgotPasswordHeading()          // → string, for navigation assertion
}
```

Design rules visible in the signatures: **readers return, specs assert** — `getErrorMessage()` returns a string; the spec does the comparison. The page object never imports an assertion library for business outcomes. **Composite + atomic both exposed** — `loginAs()` serves 90% of specs; atomic methods exist for the negative matrix.

1.3 Locator strategy — Login page

ELEMENT	LOCATOR (PLAYWRIGHT)	TIER	RATIONALE
Username field	getByPlaceholder('Username') — fallback [name="username"]	1 → 2	Placeholder is user-visible; name attr is stable
Password field	getByPlaceholder('Password') — fallback [name="password"]	1 → 2	Same
Login button	getByRole('button', {name:'Login'})	1	Role + accessible name
Invalid-credentials banner	getByRole('alert')	1	The error renders as an alert component
Required-field messages	.oxd-input-group :text("Required") scoped per field group	4 (scoped)	No semantic hook exists; class scoped to the input group
Forgot password link	getByText('Forgot your password?')	3	Stable visible text

1.4 Data flow — Login

Credentials originate in fixtures (valid) or the invalid-credential matrix (data-driven loop over the matrix — one logical test, N data rows). Secrets path: `utils/env.js` reads `OHRM_USERNAME` / `OHRM_PASSWORD` from env with fixture fallback, so CI can inject secrets without code change. Post-login, the spec asserts the handoff by constructing `DashboardPage` and checking `isLoaded()` — `LoginPage` does not know about the dashboard.

1.5 Test cases implemented (traceability)

`TC-LOG-001` valid login → dashboard · `TC-LOG-002..005` invalid matrix (wrong pass / wrong user / both / empty) · `TC-LOG-006` required-field messages · `TC-LOG-007` forgot-password navigation · `TC-LOG-008` logout returns to login · `TC-LOG-009` unauthenticated deep-link to `/pim` redirects to login.

2. PIM module

2.1 Page object class structure

```
BasePage
 └── PIMListPage          – employee list: search, grid, delete
 └── AddEmployeePage      – the Add Employee form (create flow)
 └── EmployeeDetailsPage  – Personal Details tab of an existing employee (edit flow)
```

Three classes because PIM has three genuinely distinct screens with distinct responsibilities; collapsing them into one "PIMPage" was rejected — it would exceed ~25 methods and mix search-grid logic with form logic.

2.2 Method signatures

```
class PIMListPage extends BasePage {
  async goto()
  async searchByName(employeeName)
  async searchById(employeeId)
  async resetSearch()
  async getRecordCountText()            // → "(1) Record Found"
  async getRowByName(employeeName)      // → row handle/locator or null
  async isEmployeeListed(employeeName)  // → boolean
  async openEmployee(employeeName)
  async deleteEmployee(employeeName)
  async getToastMessage()               // → string
  async clickAddEmployee()
}
class AddEmployeePage extends BasePage {
  async fillName({ firstName, middleName, lastName })
  async setEmployeeId(employeeId)
  async toggleCreateLoginDetails(enabled)
  async fillLoginDetails({ username, password, confirmPassword, status })
  async save()
  async getValidationMessages()         // → string[]
}
class EmployeeDetailsPage extends BasePage {
  async isLoaded()                      // → boolean
  async getHeaderName()                 // → string
  async updatePersonalDetails({ ...fields })
  async save()
  async getToastMessage()               // → string
  async getFieldValue(fieldLabel)       // → string
}
```

2.3 Locator strategy — PIM

ELEMENT	LOCATOR APPROACH	TIER	NOTES
PIM menu item	getByRole('link', {name:'PIM'}) in the left sidebar	1	Sidebar links have accessible names
Employee Name search	getByPlaceholder('Type for hints...') scoped to the "Employee Name" group	1 (scoped)	Two identical type-ahead fields exist — must scope by labelled group, the module's known trap
Employee ID search	input inside the group labelled "Employee Id"	2 (scoped)	
Search / Reset buttons	getByRole('button', {name:'Search'}) etc.	1	
Grid rows	.oxd-table-card filtered by hasText: employeeName	4 (scoped + filtered)	Grid renders as div-cards, not a table
Row delete icon	trash-icon button within the resolved row	4 (scoped)	Never global — always within getRowByName() result
Confirm-delete dialog	getByRole('button', {name:'Yes, Delete'})	1	
Toast	.oxd-toast text	4	Short-lived — read immediately, auto-retrying assertion
Add Employee name fields	[name="firstName"], [name="middleName"], [name="lastName"]	2	Real name attributes exist here
Save buttons	getByRole('button', {name:'Save'}) scoped to the active form	1 (scoped)	Details page has multiple Save buttons — scope to the card under edit

2.4 Data flow — PIM

Key rules: Every PIM test owns its data — `uniqueEmployee()` stamps a per-run timestamp so parallel workers and other demo-site visitors can't collide. **Create-then-verify-then-delete** is the standard shape; delete failures in teardown are logged but don't fail the test (shared env may have already reset).

Objects, not positional args — `fillName({ firstName, lastName })` keeps call sites readable and lets invalid-data tests omit fields naturally. **Auth is pre-established** (storageState / `cy.session()`): PIM specs start on an authenticated session and call `PIMListPage.goto()` directly — no UI login per test.

2.5 Test cases implemented (traceability)

`TC-PIM-001` add employee (basic) · `TC-PIM-002` add employee with login details · `TC-PIM-003` add-employee validation · `TC-PIM-004` search by name · `TC-PIM-005` search by ID · `TC-PIM-006` reset search · `TC-PIM-007` edit personal details + toast + persisted value · `TC-PIM-008` delete employee + confirm + gone from grid · `TC-PIM-009` record-count text matches result rows.

---

3. Shared LLD conventions (both modules)

1. **Constructor contract:** every page object takes the driver handle (`page` in Playwright; nothing in Cypress — `cy` is ambient) and defines all locators in the constructor or as getters — never inline inside methods.
2. **Waiting:** actions end when the UI is ready for the next step (e.g., `save()` resolves after the toast or navigation), so specs never insert waits.
3. **Return-type discipline:** action methods return `void`/`this`; reader methods return primitives or plain objects; nothing returns raw locators to specs.
4. **Naming:** `goto*` navigation, `get*` readers, `is*` booleans, verbs for actions — identical names across the Cypress and Playwright implementations to keep the parity review mechanical.
